

Reneonix Website Technical Guide

This is the working technical reference for the Reneonix website. It explains what the site contains, how it works, why the current implementation is structured this way, and how to maintain it.

The guide is written from the source code in this repository. Statements under **Current behavior** are observations of the code. Statements under **Why this approach** describe the engineering reason the approach makes sense; they are not claims about the original author's private decision process.

1. What This Website Is

Reneonix presents itself as a B2B industrial deep-tech company for material circularity. The central business message is that waste is not only a collection problem: it is also an identification, quality, traceability, compliance, and materials-engineering problem.

The website communicates one integrated platform with three layers:

- **Hardware:** vision inspection, AI sorting, sensors, actuators, and recovery machinery.
- **Software:** TraceOS/Retraiz material traceability, dashboards, verification, reporting, EPR support, and material passports.
- **Material science:** testing, formulation, validation, and conversion of recovered waste into useful industrial materials.

The intended audiences are industrial buyers, glass manufacturers, recycling operators, beverage and FMCG brands, municipalities, government bodies, investors, partners, researchers, job candidates, and journalists.

The site is therefore both:

1. A marketing and credibility surface that explains the company. 2. A technical product story that demonstrates how the platform works. 3. A lead-generation system for contact, investment, partnerships, and careers. 4. A content system containing industry pages, blog articles, policy information, and downloadable article PDFs.

2. Technology Stack

Runtime technologies

- **React 18** renders the interface as reusable components.
- **React DOM** attaches React to the HTML element with `id="root"`.
- **Vite** provides the development server and production bundler.
- **JavaScript/JSX** is used instead of TypeScript. JSX is JavaScript syntax that lets components describe HTML-like UI.
- **CSS** is split between a shared global stylesheet and page/component stylesheets.

Libraries

- **lucide-react** supplies interface icons as React components.
- **GSAP and ScrollTrigger** drive complex scroll-based storytelling and pinned sections.
- **Three.js and OGL** support advanced visual effects; Three.js is code-split because it is large. OGL is currently listed as a dependency but should be checked before future cleanup.

- **@react-pdf/renderer** generates article PDFs in the browser/application bundle.
- **@emailjs/browser** sends the main contact form from the browser.
- **@web3forms/react** sends the investor inquiry form.
- **Playwright** is installed for browser automation, but there is currently no npm test script using it.

Why these technologies fit the site

React fits a site with many repeated patterns, interactive controls, and route-specific pages. Vite gives fast local development and a straightforward static deployment. GSAP is used where CSS transitions are not enough: for synchronized timelines, pinned scroll scenes, and imperative animation of many elements. Lucide avoids manually maintaining many icon SVGs. Third-party form services avoid the need to build and deploy a custom server just to receive inquiries.

The tradeoff is complexity: animation libraries, video, WebGL, PDF generation, and third-party integrations increase bundle size, browser work, and maintenance risk.

3. Project Layout

Important files and directories:



4. How the Application Starts

The browser first receives `index.html`. It contains a `<div id="root"></div>` but almost no page body content. The module script loads `src/main.jsx`.

`src/main.jsx` does three things:

1. Imports the global stylesheet.
2. Sets `history.scrollRestoration = 'manual'` where supported. This stops the browser from restoring an old scroll position while animated pages are recalculating their layout.
3. Calls `createRoot(...).render(...)` to mount `<App />` inside React `StrictMode`.

`StrictMode` is a development aid. It intentionally exercises some lifecycle paths more than once in development to expose unsafe side effects. It does not render a second production application.

`App.jsx` then mounts the stable site shell:



The home page is composed of `Hero` , `Brands` , `Solutions` , `About` , `Investors` , `Testimonials` , `Highlights` , and `CtaBanner` . Other pages are selected from the route and loaded lazily.

5. Routing Without React Router

This project does not use React Router. It implements a small router directly with the browser History API.

Route calculation

`getRoute()` in `src/App.jsx` reads `window.location.pathname` and converts a URL into an internal route name such as `home` , `solutions` , `hardware` , or `blog-article-mrm` .

The important route families are:

- `/` `home`
- `/solutions` `solutions overview`
- `/solutions/hardware` `hardware systems`
- `/solutions/software` `TraceOS/Retraiz software`
- `/solutions/material-science` `material science`
- `/industries` `industry overview`
- `/industries/glass-manufacturers`
- `/industries/mining-minerals`
- `/industries/beverage-fmcg-brands`
- `/industries/recycling-operators`
- `/industries/municipalities-government`
- `/careers`
- `/investors-page`
- `/blog` and five article paths
- `/contact-us`
- `/policy`

Unknown paths currently fall back to the home page. There is no dedicated 404 page.

Navigation flow

`src/utils/nav.js` exports two helpers:

- `navigate(path)` calls `history.pushState()` and then manually dispatches a `PopStateEvent` .
- `navClick(path)` returns an anchor click handler that prevents a full browser reload and calls `navigate()` .

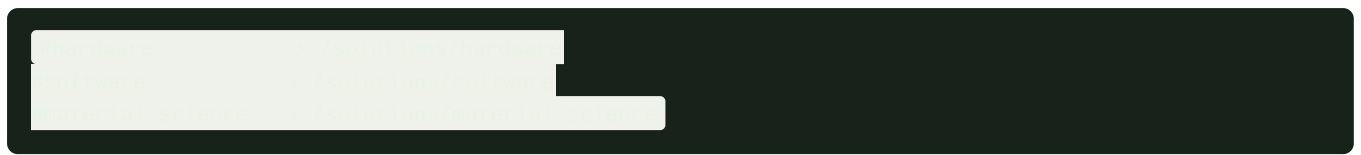
`pushState()` changes the address bar but does not notify React by itself. The synthetic `popstate` event is the bridge that tells `App` to calculate the new route and update its state. Real browser Back and Forward buttons also produce `popstate`, so the same listener handles both cases.

Why use this instead of React Router

For a mostly static marketing site with a known set of routes, this is a small dependency-free router. It keeps navigation client-side and preserves the existing URL structure. The cost is that the project must manually handle route matching, scroll resets, metadata, 404 behavior, and server fallback rules. React Router would provide conventions and nested routing, but would add another framework layer to a site that currently has a limited route model.

Legacy hash support

Older links used fragments such as `/#hardware`. `redirectLegacyHash()` maps old hashes to current real paths, for example:



Home section hashes such as `#about` and `#investors` remain section anchors. This preserves old shared links while allowing real pages to have distinct URLs and canonical metadata.

Cross-page section jumps

A link can target a section on another page. The source stores the target ID in `sessionStorage` under `sw_scroll_target`, navigates to the destination, and `App.jsx` retries finding the target after the lazy page mounts. It then scrolls to the element while subtracting the sticky header height.

This is necessary because the destination DOM does not exist until the new route's component has loaded and rendered.

6. Lazy Loading and Code Splitting

Sub-pages are declared with `React.lazy(() => import(...))`. The route page is downloaded only when needed. The page is rendered inside `Suspense`.

This reduces the initial JavaScript needed for the home page. The production build confirms that Vite creates separate chunks for many pages and libraries.

The current `Suspense` fallback is `null`, meaning there is no visible loading indicator while a lazy page downloads. This produces a clean transition but can look blank on a slow connection. A small route loading state would improve feedback if this becomes noticeable.

7. Shared Visual System

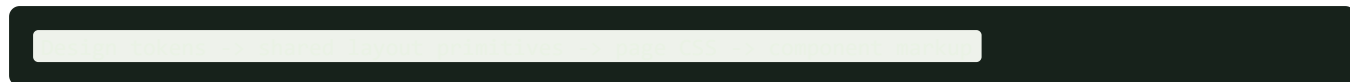
`src/styles/global.css` defines the design language:

- Dark base colors and charcoal panels.
- Lime accent colors for action and emphasis.
- Light paper sections for contrast.

- Inter for body/interface text, Arimo for headings, Montserrat and JetBrains Mono for specialized use.
- CSS custom properties for colors, spacing, radii, gutters, maximum widths, and section padding.
- Shared `.container`, `.section`, `.btn`, `.eyebrow`, navigation, footer, reveal, and responsive rules.

Page CSS files are imported by their corresponding components. This keeps page-specific layout near page-specific JSX while `global.css` owns rules used across the whole application.

A useful mental model is:



The `container` class creates consistent horizontal gutters. The section classes control the alternating dark/light rhythm. Button classes provide common interaction and accessibility styling.

8. Home Page Sections

Hero

`Hero.jsx` uses a muted, inline video as a decorative background. It does not use the native `loop` attribute because the code wants to pause briefly on the final frame before restarting. The `ended` and `timeupdate` listeners implement that behavior.

The hero headline establishes the category: “Material Circularity.” The paragraph explains the technology stack and the desired business outcome: quality-controlled, traceable, industry-ready materials. The CTA navigates to `/solutions`.

Brands

`Brands.jsx` provides social proof through brand or ecosystem logos. Its role is to reduce perceived risk after the hero makes a broad claim.

Solutions

The home `Solutions.jsx` explains the three-layer system in sequence: hardware, software, and material science. Each layer has a visual, feature list, and route CTA. `ZzRow` adds viewport-based reveal behavior.

About

`About.jsx` explains company origin, operating model, and values. It gives context that product pages alone cannot provide.

Investors, testimonials, highlights, and CTA

These sections provide credibility, human proof, external press/video references, and a final conversion path. The footer repeats the overall navigation and company identity.

9. Product and Industry Pages

Solutions overview

`SolutionsPage.jsx` gives the platform-level explanation. It includes three solution cards, a traditional-versus-Reneonix process comparison, measurable claims, industry links, and proof-oriented content.

Hardware

`HardwareSystems.jsx` contains the most technical visual narrative. A scroll-driven “follow the bottle” scene advances through scan, detect, classify, sort, and recover. The code calculates progress from scroll position, moves SVG elements, updates labels and logs, and animates route segments.

GSAP `ScrollTrigger` pins the scene on desktop. Mobile uses a separate interaction strategy where appropriate. The page kills old `ScrollTrigger` instances on mount/unmount because this is a single-page app and page transitions otherwise risk leaving animations from the previous route alive.

Software

`SoftwarePage.jsx` explains the material-data lifecycle: capture, verify, trace, analyze, and report. Desktop uses a pinned, scrubbed GSAP timeline; mobile uses a horizontal swipeable track with CSS scroll snapping. This is a deliberate responsive difference: scroll-jacking is more difficult and uncomfortable on small touch screens.

`SoftwarePagePreview.jsx` supplies the dashboard-style product mockup.

Material science

`MaterialSciencePage.jsx` presents the R&D and validation layer, connecting recovered streams to formulations and industrial applications.

Industries

`IndustriesPage.jsx` acts as a directory. Its cards state the maturity signal for each market: live deployment, pilot discussion, positioning, or concept. `MagicBento` adds spotlight, border glow, particles, and click interaction.

Each industry detail page then translates the same platform into a specific buyer problem and workflow.

10. Shared Interactions and Animation

Scroll reveal

`SiteEffects.jsx` handles global `.reveal`, `.reveal-img`, and `.stagger` elements. It uses `useLayoutEffect` to reveal elements already visible before the first paint, preventing a flash of invisible content. It then uses `IntersectionObserver` to reveal below-fold elements as they enter the viewport.

Several pages have local observers because lazy-loaded components mount after global effects may already have run. This is why pages such as software, hardware, and industries contain their own page-specific reveal setup.

Parallax

Elements with `data-parallax` are queried after each route change. Scroll events are throttled through `requestAnimationFrame`, which limits style updates to roughly one browser paint cycle instead of running unrestricted code for every raw scroll event.

GSAP

GSAP is used for coordinated timelines and `ScrollTrigger` pinning. It is appropriate when several elements must stay synchronized with one continuous progress value. It must be cleaned up on route changes to avoid stale triggers and memory leaks.

Splash cursor and visual effects

`SplashCursor.jsx`, `MagicRings.jsx`, and related components provide richer pointer/canvas/WebGL effects. These are decorative enhancements, not the core content. The content must remain understandable if effects are disabled or unsupported.

Reduced-motion handling exists in several places, especially global effects and video playback, but it should be reviewed consistently across all page-specific animations.

Preloader and back-to-top

`Preloader.jsx` covers the first paint while the visual system starts. `BackToTop.jsx` provides a persistent return-to-top interaction. These improve perceived polish, but a preloader also delays access to content and should not become longer than the value it provides.

11. Search

`SiteSearch.jsx` is a command-palette-style search interface. `src/data/searchIndex.js` contains the records that can be searched, including pages, sections, and articles.

The search supports keyboard interaction such as Ctrl/Cmd+K and can navigate directly to a route or a section. Because the index is manually maintained, adding a page requires adding a matching search record if it should appear in results.

12. Forms and External Services

Contact form

`ContactPage.jsx` sends inquiries through EmailJS. It dynamically loads and explicitly renders Google reCAPTCHA. It has:

- Controlled React form state.
- Required and length validation.
- A hidden honeypot field for simple bots.
- CAPTCHA validation.
- Sending, success, and error states.
- CAPTCHA reset after each attempt.

There is no custom server in this repository. The browser calls EmailJS directly.

Investor form and guide

`InvestorsPage.jsx` uses Web3Forms. The “AI guide” is a local scripted questionnaire implemented with React state and timers; it is not a live AI model or backend conversation. Answers are collected and then connected to the investor inquiry flow.

Careers

`Careers.jsx` uses email links and Gmail compose links rather than uploading resumes to a backend. This is simple to deploy but less structured than a recruitment system.

Security model

Public browser keys are visible in the shipped JavaScript by design for services that require browser-side identifiers. They are not a substitute for server-side secrets. Do not place private API keys, database credentials, or privileged tokens in React source or `public/`.

For production hardening, consider moving sensitive submission logic behind a serverless function, adding rate limiting, and reviewing third-party service privacy and retention settings.

13. SEO and Metadata

`index.html` supplies fallback metadata for crawlers and link previews that do not execute JavaScript. It also contains Organization JSON-LD, Google Analytics, font loading, preconnects, and asset preloads.

`src/data/pageMeta.js` contains route-specific titles, descriptions, Open Graph images, Twitter images, canonical URLs, and article dates. `App.jsx` updates these tags whenever the internal route changes.

This gives users and JavaScript-capable crawlers accurate metadata per page. The limitation is that a purely static scraper may still see only the home metadata. Server-side rendering or pre-rendered static pages would improve SEO certainty for every route.

`public/sitemap.xml` and `public/robots.txt` support crawling. Keep the sitemap synchronized with every public route.

14. Assets and Performance

Assets in `public/` are served from the site root. A file named `public/example.png` is referenced as `/example.png`. Spaces in filenames work in the browser but are easy to mishandle in URLs and metadata, so future assets should use lowercase names with hyphens.

The site uses large videos, images, SVG scenes, WebGL effects, GSAP, Three.js, and PDF rendering. The production build currently succeeds, but Vite reports large chunks, especially the PDF renderer and Three.js vendor code.

Current performance techniques include:

- Lazy-loaded route components.
- Lazy-loaded below-fold images.
- `decoding="async"` on many images.
- `fetchpriority="high"` and preload for key hero assets.
- Manual Vite chunks for major libraries.
- `requestAnimationFrame` throttling for parallax.
- Pausing some videos when they leave the viewport.

Performance improvements to consider later:

- Do not load PDF rendering code until a PDF is requested.
- Review whether every global visual effect is necessary on every route.
- Compress and dimension large images and videos.
- Use a visible but small `Suspense` loading state.
- Test on a mid-range mobile device, not only a desktop browser.

15. Deployment

`netlify.toml` tells Netlify to run `npm run build` and publish `dist/`.

`public/_redirects` is important for this manual SPA router. It sends unknown server requests to `/index.html` with status 200, allowing the React application to inspect paths such as `/solutions/hardware` after a direct refresh.

Without the fallback, internal navigation might work from the home page but refreshing a nested URL could return a server 404.

Useful commands:



There are currently no `test` or `lint` scripts in `package.json`.

16. How to Add or Change a Page

1. Create the page component in `src/components/`. 2. Create its page stylesheet if it needs page-specific rules. 3. Import the component with `lazy()` in `src/App.jsx`. 4. Add an exact route branch in `getRoute()` before any broader prefix branch that could catch it. 5. Render the page in the route-selection block. 6. Add the path to the navigation data in `Header.jsx`, `Footer.jsx`, or both. 7. Add a record to `src/data/searchIndex.js` if it should be searchable. 8. Add title, description, and image metadata to `src/data/pageMeta.js`. 9. Add the path to `public/sitemap.xml` if it is public and indexable. 10. Add a link from a relevant page and test direct loading, client-side navigation, Back, Forward, mobile navigation, and refresh. 11. Run `npm run build`.

For a new section on an existing page, use a stable `id`, update any cross-page section link, and check the sticky-header offset behavior.

17. Maintenance Checklist

Before deploying a meaningful change:

- Run `npm run build`.
- Open the changed route directly, not only by navigating from home.
- Test browser Back and Forward.
- Test the mobile menu and dropdowns.
- Test reduced motion if the change adds animation.
- Check keyboard focus and form error states.
- Check the browser console for failed assets or third-party calls.
- Check desktop and narrow mobile layouts for overflow.
- Update `pageMeta.js`, `searchIndex.js`, `sitemap`, and this guide when the public information architecture changes.
- Confirm forms still submit in the production environment.
- Review external URLs and image paths.

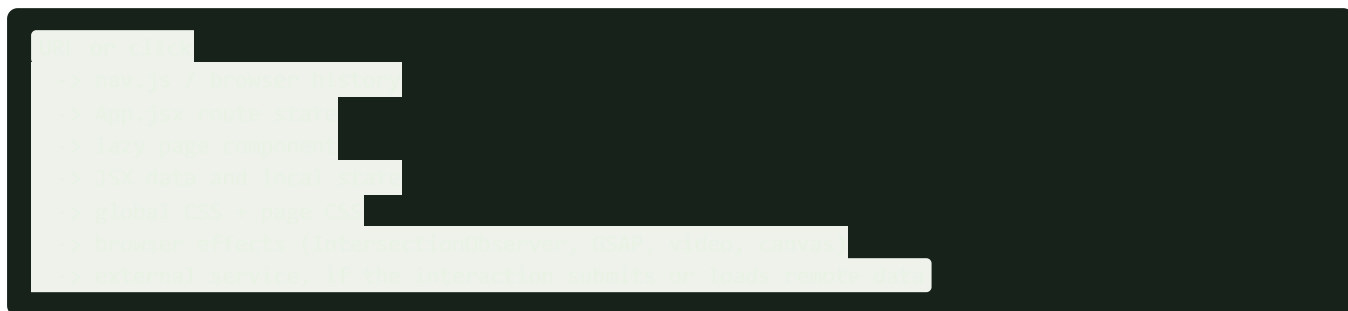
18. Known Inconsistencies to Resolve

These are maintenance findings from the current source, not assumptions:

- The old `README.md` describes an earlier hash-based structure and no longer matches the current route/page inventory.
- `pageMeta.js` references `/solutions-hero.jpeg`, while the visible asset inventory includes `solutions-hero.png`; verify the Open Graph image path.
- TraceOS, Retraiz, and RETRIAZ are used in different places. Decide whether these are distinct products, aliases, or a naming error, then standardize public copy.
- Several resource links are intentionally inactive placeholders, including Case Studies, Whitepapers, Sustainability Reports, Press & Media, and FAQ.
- The municipalities/government route should be checked against `public/sitemap.xml`.
- Unknown paths render the home page instead of a 404 page.
- Route metadata is client-rendered after JavaScript starts; consider pre-rendering if organic search is a priority.
- `ogl` and the `.env.example` Resend entry appear to need review because their current usage is unclear.
- The build reports large chunks; PDF and visual-effect loading are good candidates for later optimization.

19. The Simplest Mental Model

When you are trying to understand a change, follow this chain:



React owns the component tree and state. The browser owns the URL, History API, DOM, media, and observers. CSS owns most layout and simple animation. GSAP owns synchronized imperative animation. Netlify owns the build and fallback delivery. External providers own form delivery, CAPTCHA, analytics, fonts, and some remote thumbnails.

That division of responsibility is the key to debugging the site: first identify which layer owns the behavior, then inspect the smallest relevant file.